

Complete Data Engineering Learning Path with NASA APIs

Project Overview

Build a comprehensive data pipeline using multiple NASA APIs (NEOs, APOD, Mars Rovers, Earth Observatory) to learn all core Data Engineering concepts from scratch.

Phase 1: Foundation & Data Ingestion (Weeks 1-2)

Concepts You'll Learn

- API interactions & authentication
- Data extraction patterns
- Error handling & retry logic
- Logging & monitoring basics
- Environment management

What to Build

Simple API Ingestion Script

1. Setup Your Environment

- Install Python 3.9+
- Create virtual environment: `python -m venv venv`
- Install: `requests`, `python-dotenv`, `pandas`
- Get free NASA API key from <https://api.nasa.gov>

2. Build Your First Extractor

Project Structure:

```
nasa-de-project/  
├── config/  
│   └── config.yaml  
├── extractors/  
│   ├── __init__.py  
│   ├── base_extractor.py  
│   └── neo_extractor.py  
├── .env  
└── main.py
```

3. Tasks to Complete

- Create a base class for API calls with retry logic
- Extract NEO data for date ranges
- Save raw JSON responses to local files
- Implement proper error handling
- Add logging (use Python's `logging` module)

4. Key Learning

- Understand REST APIs
- Handle rate limiting
- Manage API keys securely
- Structure your codebase

Phase 2: Data Storage & Organization (Weeks 3-4)

Concepts You'll Learn

- File system organization (Data Lake concept)
- Database design & normalization
- SQL fundamentals
- Data partitioning strategies
- Schema design

What to Build

Local Data Lake + SQLite Database

1. Organize Raw Data (Bronze Layer)

```
data/
├── raw/
│   ├── neos/
│   │   ├── 2024-01/
│   │   │   ├── 2024-01-01.json
│   │   │   └── 2024-01-02.json
│   ├── apod/
│   └── mars_rover/
```

2. Design Your Database Schema

- Create SQLite database
- Design tables: `asteroids`, `close_approaches`, `api_metadata`
- Implement proper primary keys and foreign keys
- Add indexes for query performance

3. Tasks to Complete

- Write SQL DDL scripts for table creation
- Create a database connection manager
- Insert data from JSON into SQLite
- Write queries to analyze the data
- Implement incremental loading (don't reload same dates)

4. Key Learning

- Database normalization (1NF, 2NF, 3NF)
 - SQL CRUD operations
 - Transaction management
 - Query optimization basics
-

Phase 3: Data Transformation & Quality (Weeks 5-6)

Concepts You'll Learn

- ETL vs ELT patterns
- Data cleaning & validation
- Data type conversions
- Handling missing data
- Data quality checks
- Idempotency

What to Build

Transformation Pipeline (Silver Layer)

1. Create Transformation Scripts

```
transformers/  
├── __init__.py  
├── base_transformer.py  
├── neo_transformer.py  
└── data_quality.py
```

2. Tasks to Complete

- Parse nested JSON structures
- Convert data types (dates, numbers)
- Handle missing values
- Standardize units (km to AU, etc.)
- Calculate derived metrics (velocity in different units)
- Create dimension tables (lookup tables for asteroid names)
- Implement data quality checks:
 - Null checks
 - Range validation
 - Duplicate detection
 - Schema validation

3. Key Learning

- Pandas transformations
 - Data type handling
 - Error handling in transformations
 - Writing unit tests for transformations
-

Phase 4: Workflow Orchestration (Weeks 7-8)

Concepts You'll Learn

- DAGs (Directed Acyclic Graphs)
- Task dependencies
- Scheduling & cron expressions
- Backfilling data
- Workflow monitoring
- Idempotent pipelines

What to Build

Airflow Pipeline (or Prefect/Dagster)

1. Setup Apache Airflow

- Install Airflow locally
- Initialize database: `airflow db init`
- Create admin user
- Start web server and scheduler

2. Create Your First DAG

```
dags/  
├── nasa_daily_pipeline.py  
└── nasa_backfill_pipeline.py
```

3. DAG Structure

4. Tasks to Complete

- Create daily scheduled DAG (runs every day at 6 AM)
- Implement task dependencies
- Add retry logic and alerts
- Create a backfill DAG for historical data
- Set up email notifications on failure
- Add sensor for API availability

5. Key Learning

- DAG design patterns
 - Cron expressions
 - XComs for passing data between tasks
 - Backpressure handling
 - Monitoring workflows
-

Phase 5: Cloud Migration (Weeks 9-10)

Concepts You'll Learn

- Cloud storage (S3, GCS, Azure Blob)
- Cloud databases
- Infrastructure as Code
- Cost optimization
- Security & access management

What to Build

AWS-Based Data Pipeline (or GCP/Azure)

1. Setup AWS Infrastructure

- Create S3 buckets (raw, processed, curated)

- Setup RDS PostgreSQL instance (or Aurora)
- Configure IAM roles and policies
- Setup VPC and security groups

2. Migrate Your Pipeline

Local Files → S3

SQLite → RDS PostgreSQL

Local Airflow → AWS MWAA (Managed Airflow)

3. Tasks to Complete

- Use `boto3` for S3 operations
- Implement partitioned writes to S3 (year/month/day)
- Migrate database to PostgreSQL
- Update connection strings
- Deploy Airflow DAGs to cloud
- Set up CloudWatch logging

4. Key Learning

- AWS S3 best practices
- Database migration strategies
- IAM and security
- Cost monitoring

Phase 6: Big Data Processing (Weeks 11-12)

Concepts You'll Learn

- Distributed computing
- Spark fundamentals
- Parallel processing
- Memory management
- Optimizing transformations

What to Build

PySpark Transformation Pipeline

1. Setup Spark

- Install PySpark locally
- Configure Spark session
- Later: Use AWS EMR or Databricks

2. Rewrite Transformations in Spark

```
spark_jobs/  
├── neo_processing.py  
├── aggregations.py  
└── ml_features.py
```

3. Tasks to Complete

- Read JSON from S3 using Spark
- Perform transformations using DataFrame API
- Write partitioned Parquet files
- Implement aggregations:
 - Daily NEO counts
 - Hazardous asteroid statistics
 - Distance calculations
- Optimize with caching and partitioning
- Compare performance: Pandas vs Spark

4. Key Learning

- Lazy evaluation
 - Transformations vs Actions
 - Partitioning strategies
 - Parquet file format
 - Broadcast joins
-

Phase 7: Data Warehouse & Analytics (Weeks 13-14)

Concepts You'll Learn

- Star schema / Snowflake schema
- OLTP vs OLAP
- Data modeling for analytics
- Incremental loading (SCD Type 2)
- BI tool integration

What to Build

Dimensional Data Warehouse

1. Choose Your Warehouse

- Start with PostgreSQL with extensions
- Later: Snowflake, BigQuery, or Redshift

2. Design Star Schema

Fact Tables:

- fact_close_approaches
- fact_daily_observations

Dimension Tables:

- dim_asteroid
- dim_date
- dim_observatory

3. Tasks to Complete

- Design dimension and fact tables
- Implement SCD Type 2 for changing attributes
- Create staging tables
- Build incremental load process
- Write analytical queries:
 - Monthly trends
 - Hazardous asteroid patterns

- Size distribution analysis
- Create views for common queries
- Connect to visualization tool (Metabase/Superset)

4. Key Learning

- Dimensional modeling
 - Slowly Changing Dimensions
 - Aggregation strategies
 - Indexing for analytics
 - Materialized views
-

Phase 8: Real-Time Processing (Weeks 15-16)

Concepts You'll Learn

- Stream processing concepts
- Message queues
- Event-driven architectures
- Real-time vs batch processing
- Windowing operations

What to Build

Real-Time Alert System

1. Setup Streaming Infrastructure

- Apache Kafka (or AWS Kinesis, GCP Pub/Sub)
- Redis for caching
- WebSocket server for real-time dashboard

2. Architecture

NASA API → Kafka Producer → Kafka Topic →
→ Spark Streaming → Process → Alert Service →
→ Redis Cache → Dashboard

3. Tasks to Complete

- Create Kafka producer to poll NASA API
- Implement Kafka consumer with Spark Streaming
- Detect anomalies (unusually close approaches)
- Send alerts via email/SMS
- Cache latest data in Redis
- Build real-time dashboard (optional: Streamlit/Dash)

4. Key Learning

- Kafka basics (producers, consumers, topics)
 - Stream processing patterns
 - Stateful processing
 - Exactly-once semantics
 - Watermarking
-

Phase 9: Data Quality & Testing (Weeks 17-18)

Concepts You'll Learn

- Unit testing for data pipelines
- Integration testing
- Data validation frameworks
- Data lineage
- Monitoring & alerting

What to Build

Comprehensive Testing & Monitoring

1. Testing Framework

```
tests/
├── unit/
│   ├── test_extractors.py
│   └── test_transformers.py
├── integration/
│   └── test_pipeline.py
└── data_quality/
    └── test_data_rules.py
```

2. Tasks to Complete

- Write unit tests with `pytest`
- Create data quality tests using Great Expectations
- Implement data profiling
- Set up data lineage tracking
- Create monitoring dashboards (Grafana)
- Set up alerts (PagerDuty/Slack)
- Document data catalog

3. Tools to Use

- Great Expectations for data validation
- `pytest` for unit/integration tests
- Prometheus + Grafana for monitoring
- dbt (data build tool) for documentation

4. Key Learning

- Test-driven development for data
 - Data contracts
 - SLA monitoring
 - Incident response
-

Phase 10: Advanced Topics & ML Integration (Weeks 19-20)

Concepts You'll Learn

- Feature engineering for ML
- MLOps basics
- Model serving
- A/B testing
- Data versioning

What to Build

ML-Powered Predictions

1. Feature Engineering Pipeline

- Extract features from asteroid data
- Create time-based features
- Build feature store (using Feast or Tecton)

2. ML Model

- Predict asteroid hazard level
- Forecast close approach dates
- Cluster similar asteroids

3. Tasks to Complete

- Build feature extraction pipeline
- Train model with historical data
- Version models (MLflow)
- Deploy model API (FastAPI)
- Integrate predictions into pipeline
- Monitor model performance
- Retrain automatically

4. Key Learning

- Feature stores
- Model versioning

- A/B testing infrastructure
 - Model drift detection
-

Additional NASA APIs to Expand Project

Once comfortable, add these data sources:

1. APOD (Astronomy Picture of the Day)

- Daily image ingestion
- Image metadata extraction
- Computer vision on images

2. Mars Rover Photos

- Multi-rover data collection
- Image storage optimization
- Time series analysis

3. Earth Observatory (EONET)

- Natural events tracking
- Geospatial analysis
- Correlation with NEO data

4. Exoplanet Archive

- Complex nested data structures
 - Scientific computing
 - Statistical analysis
-

Complete Tech Stack Summary

Languages: Python, SQL

Orchestration: Apache Airflow (or Prefect/Dagster)

Storage:

- Object Storage: AWS S3 / GCS / Azure Blob

- Databases: PostgreSQL, SQLite
- Warehouse: Snowflake / BigQuery / Redshift
- Cache: Redis

Processing:

- Batch: Pandas, PySpark
- Stream: Kafka, Spark Streaming

Testing & Quality:

- pytest, Great Expectations, dbt

Monitoring:

- Prometheus, Grafana, CloudWatch

ML/AI:

- scikit-learn, MLflow, Feast

Infrastructure:

- Docker, Terraform, GitHub Actions

Weekly Study Plan

Daily commitment: 2-3 hours **Weekend:** 4-6 hours on projects

- **Monday-Wednesday:** Learn new concepts (video tutorials, documentation)
- **Thursday-Friday:** Implement what you learned
- **Saturday:** Build and test features
- **Sunday:** Code review, refactoring, documentation

Learning Resources

1. Books

- "Fundamentals of Data Engineering" by Joe Reis
- "Designing Data-Intensive Applications" by Martin Kleppmann

2. Online Courses

- DataTalks.Club Data Engineering Zoomcamp (FREE)
- Apache Airflow Tutorial by Marc Lamberti

3. Documentation

- NASA API docs: <https://api.nasa.gov>
- Apache Airflow docs
- AWS/GCP documentation

4. Practice

- LeetCode SQL problems
- HackerRank Python challenges

Success Metrics

Track your progress:

- ☐ Pipeline runs successfully daily
- ☐ Can explain every component
- ☐ Written tests cover 80%+ of code
- ☐ Documentation is complete
- ☐ Can recover from failures automatically
- ☐ Cost of cloud resources < \$50/month
- ☐ Built a portfolio project on GitHub
- ☐ Created a blog post explaining your architecture

Final Project Ideas

Combine everything you've learned:

1. Real-time NEO Dashboard

- Live tracking of asteroids
- Historical trend analysis

- Prediction models
- Alert system

2. **Multi-source NASA Analytics Platform**

- Integrate 4+ NASA APIs
- Cross-dataset analysis
- ML predictions
- Public API for your processed data

3. **Data Quality Framework**

- Automated data validation
- Lineage tracking
- Self-healing pipelines
- Comprehensive monitoring

Next Steps After Completion

1. **Contribute to open source** data engineering projects
2. **Start a blog** documenting your learning
3. **Build a portfolio** on GitHub
4. **Get certified:** AWS Data Analytics, Databricks, Airflow
5. **Apply for Junior Data Engineer positions**

Remember: Data Engineering is learned by doing. Start small, iterate, and gradually add complexity. Good luck! 🚀